

Advanced Machine Learning

DATA 642

Lecture 14 — Autoencoders Part II

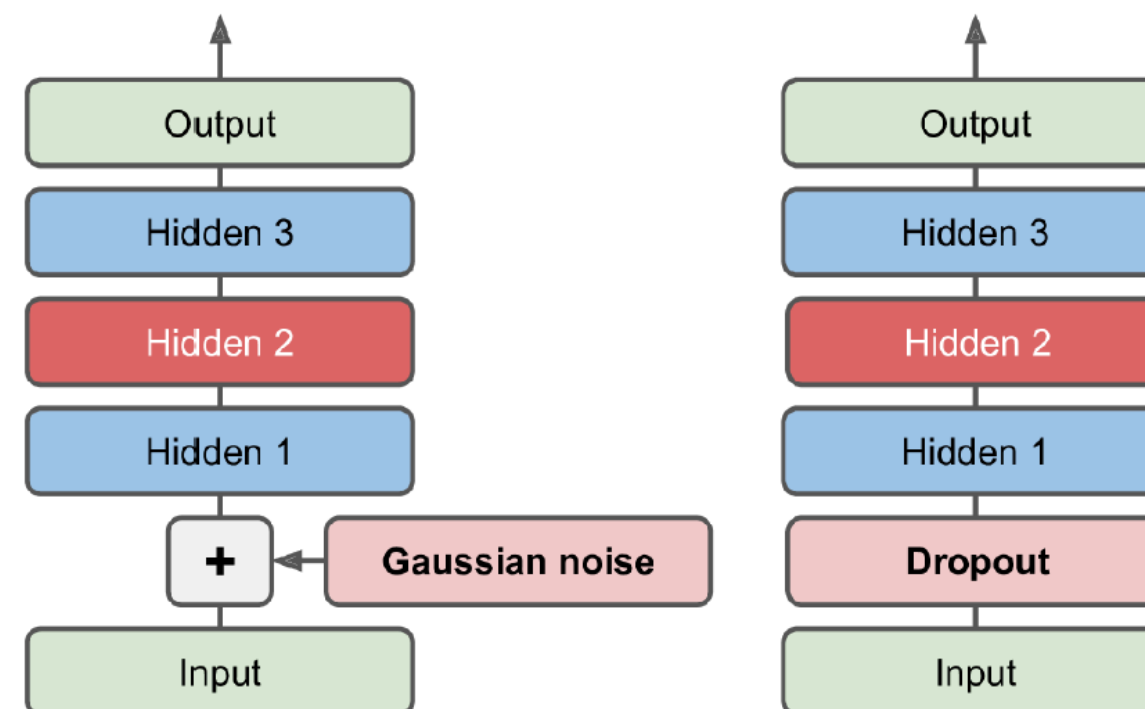
*Figures and notes are from the book Mathematics for Machine Learning by A. Aldo Faisal, Cheng Soon Ong, and Marc Peter Deisenroth, Machine Learning A Bayesian and Optimization Perspective, Sergios Theodoridis, Elsevier, and Hands-On Machine Learning with Scikit-Learn Keras and TensorFlow, Aurelien Geron, O'Reilly

Tying Weights

- Tying weights is a technique commonly employed in neatly symmetrical autoencoders, like the one we just built. It involves linking the weights of decoder layers to corresponding encoder layers. This reduces the number of weights by half, leading to faster training and mitigating the risk of **overfitting**.
- Tied weights promote information sharing between encoder and decoder layers, fostering a more cohesive learning process.
- Moreover, this approach aids in the generalization of learned features, enhancing the autoencoder's ability to reconstruct diverse input samples.

Denoising Autoencoders

- Denoising autoencoders offer a compelling approach to encourage the learning of useful features within an autoencoder framework. This technique involves adding noise to the input data and training the autoencoder to reconstruct the original, noise-free inputs.
- The implementation is straightforward: it is a regular stacked autoencoder with an additional **Dropout** layer applied to the encoder's inputs (or you could use a **GaussianNoise** layer instead). Recall that the **Dropout** layer is only active during training (and so is the **GaussianNoise** layer). Denoising autoencoders, with Gaussian noise (left) or dropout (right).



Sparse Autoencoder

- Another kind of constraint that often leads to good feature extraction is **sparsity**: by adding an appropriate term to the cost function, the autoencoder is pushed to reduce the number of active neurons in the coding layer.

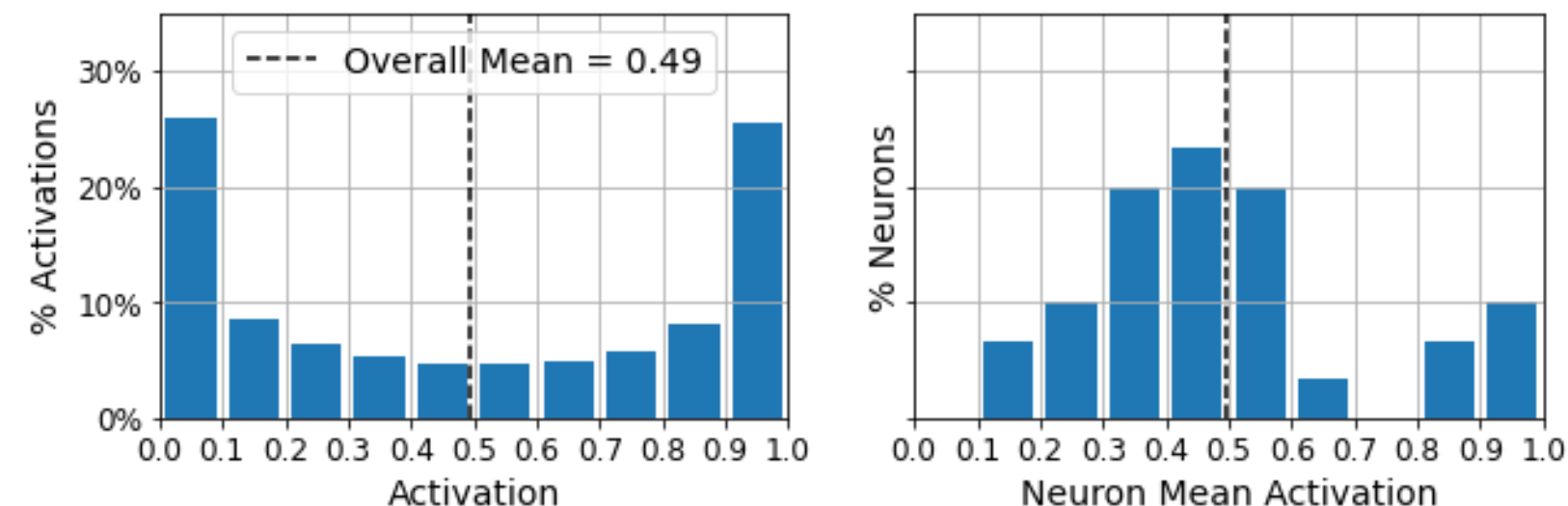
$$J(\mathbf{x}, y) = L(\mathbf{x}, y) + \lambda \cdot \Omega(h), \quad (1)$$

where $L(\mathbf{x}, y)$ represents the reconstruction loss (e.g., mean squared error), h denotes the activations of neurons in the coding layer, $\Omega(h)$ is a sparsity penalty term that measures the sparsity level of h , and λ is a hyperparameter that controls the strength of the sparsity penalty.

- As a result, each neuron in the coding layer typically ends up representing a useful feature (if you could speak only a few words per month, you would probably try to make them worth listening to).
- A simple approach is to use the sigmoid activation function in the coding layer (to constrain the codings to values between 0 and 1), use a large coding layer (e.g., with 300 units), and add some ℓ_1 regularization to the coding layer's activations (the decoder is just a regular decoder).

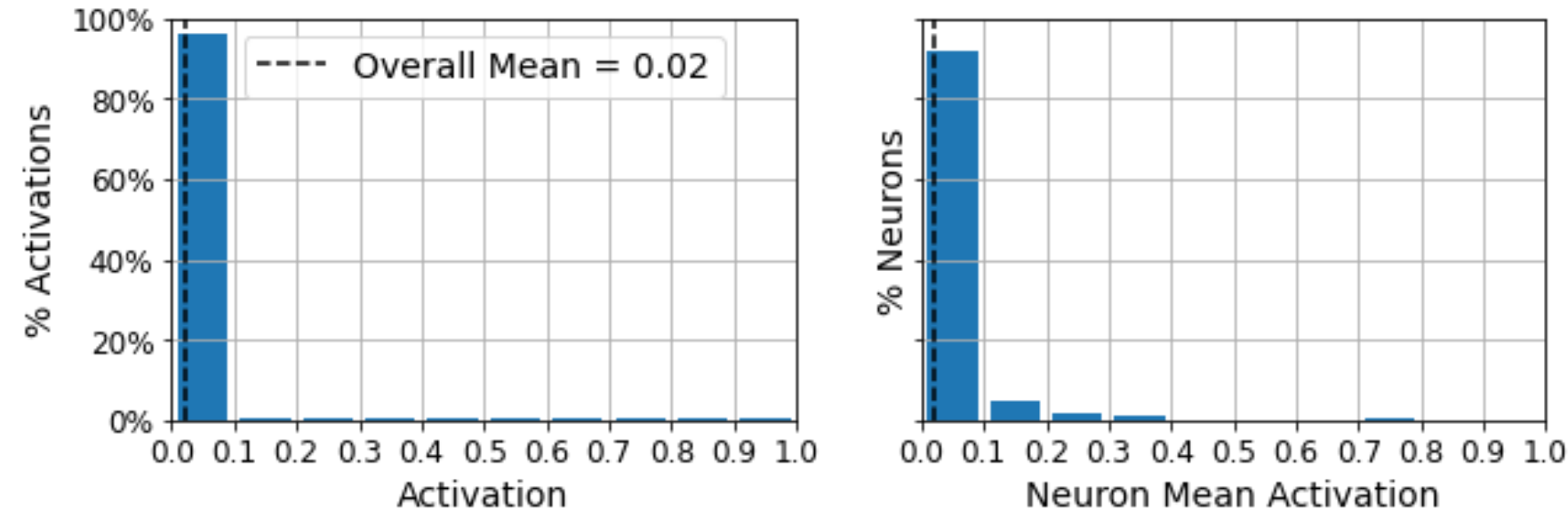
How do activations behave?

- Let's use these functions to plot histograms of the activations of the encoding layer.
 - The histogram on the left displays the distribution of all the activations. Values close to 0 or 1 are more frequent overall, consistent with the saturating nature of the sigmoid function.
 - The histogram on the right exhibits the distribution of mean neuron activations. Most neurons have a mean activation close to 0.5.
- Both histograms indicate that each neuron tends to either fire close to 0 or 1, with about 50% probability each. However, some neurons fire almost all the time. This suggests to use a penalty to better **enforce sparsity in the activations of the encoding layer**.



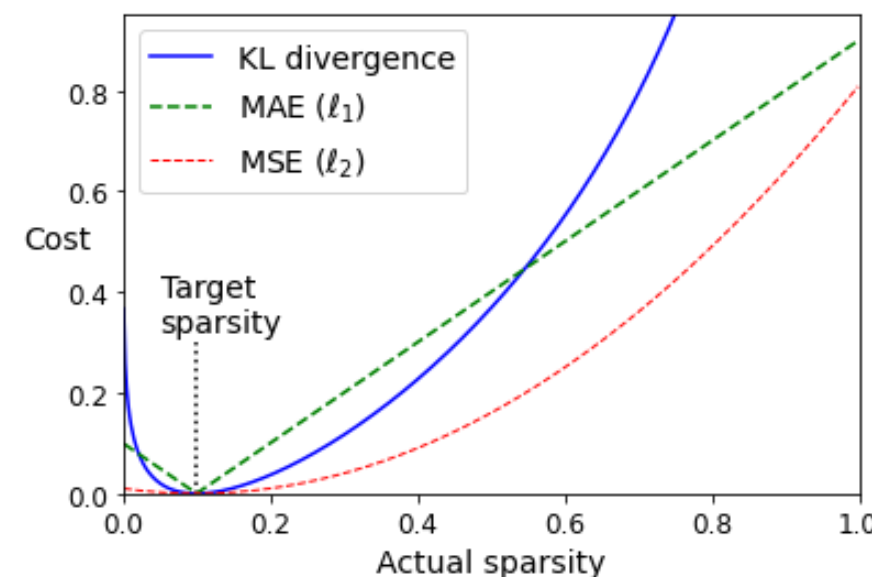
How do activations behave under the ℓ_1 penalty

- The penalty encourages the neural network to produce codings close to zero.
- However, the network also faces a penalty if it fails to accurately reconstruct the inputs.
- Thus, the network is incentivized to output at least a few non-zero values to minimize reconstruction errors.
- By using the ℓ_1 norm instead of the ℓ_2 norm:
 - The neural network is pushed to retain the most significant codings.
 - It discards those deemed unnecessary for faithfully representing the input observation.



Adding the KL divergence as a penalty function

- Another effective approach, often resulting in superior outcomes, entails assessing the actual sparsity of the coding layer during each training iteration.
- When the measured sparsity deviates from a predetermined target sparsity, the model is penalized accordingly.
- This process involves computing the average activation level of each neuron in the coding layer across the entire training batch.
- Following the determination of the mean activation per neuron, the objective shifts to penalizing neurons that exhibit excessive or inadequate activity.
- To achieve this, a sparsity loss term is integrated into the cost function such as the Kullback-Leibler (KL) divergence.
- The KL divergence boasts significantly stronger gradients compared to the mean squared error, as demonstrated in the following figure.



KL divergence penalty

Given two discrete probability distributions P and Q , the KL divergence between these distributions, is computed as follows:

$$D_{\text{KL}}(P \parallel Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)} \quad (2)$$

In our case, we want to measure the divergence between the target probability p that a neuron in the coding layer will activate and the actual probability q (i.e., the mean activation over the training batch). So the KL divergence simplifies to

$$D_{\text{KL}}(p \parallel q) = p \log \frac{p}{q} + (1 - p) \log \frac{1 - p}{1 - q} \quad (3)$$

- Once we have computed the sparsity loss for each neuron in the coding layer, we sum up these losses and add the result to the cost function. In order to control the relative importance of the sparsity loss and the reconstruction loss, we can multiply the sparsity loss by a sparsity weight hyperparameter.
- If this weight is too high, the model will stick closely to the target sparsity, but it may not reconstruct the inputs properly, making the model useless. Conversely, if it is too low, the model will mostly ignore the sparsity objective and will not learn any interesting features.

How do activations behave under the KL-divergence penalty

After training a sparse autoencoder, the activations of the neurons in the coding layer are mostly close to 0 (about 70% of all activations are lower than 0.1), and all neurons have a mean activation around 0.1 (about 90% of all neurons have a mean activation between 0.1 and 0.2), as shown in

